C++ STL

Operations · Time Complexity · Memory · Limitations

1. vector — Dynamic Array

What it does	Memory	Does NOT support
Contiguous resizable array. Random access in O(1). Doubles capacity on overflow.	O(n)	No O(1) insert/delete in the middle.

```
#include vector v; // empty vector v(n, 0); // n zeros
```

Operation	C++ Function	Time	Notes
Access element	`v[i] / v.at(i)`	O(1)	at() does bounds check
Push to back	`v.push_back(x)`	O(1) amort	Rarely O(n) on resize
Pop from back	`v.pop_back()`	O(1)	
Insert at position	`v.insert(it, x)`	O(n)	Shifts all elements right
Erase at position	`v.erase(it)`	O(n)	Shifts all elements left
Size / Empty	`v.size() / v.empty()`	O(1)	
Clear	`v.clear()`	O(n)	Destroys all elements
Sort	`sort(v.begin(),v.end())`	O(n log n)	std::sort
Binary search	`lower_bound / upper_bound`	O(log n)	Only on sorted vector
Front / Back	`v.front() / v.back()`	O(1)	

2. stack — LIFO Adapter

What it does	Memory	Does NOT support
Last-In First-Out. Only the top is accessible. Built on deque by default.	O(n)	No iteration, no random access, no search.

```
#include stack st;
```

Operation	C++ Function	Time	Notes
Push	`st.push(x)`	O(1)	
Pop	`st.pop()`	O(1)	Does NOT return value — call top() first
Top	`st.top()`	O(1)	Undefined on empty stack
Size/Empty	`st.size() / st.empty()`	O(1)	

3. queue — FIFO Adapter

What it does	Memory	Does NOT support

First-In First-Out. Push to back, pop from front.	O(n)	No random access, no iteration, no search.

```
#include queue q;
```

Operation	C++ Function	Time	Notes
Push to back	`q.push(x)`	O(1)	
Pop from front	`q.pop()`	O(1)	Does NOT return value
Front	`q.front()`	O(1)	Oldest element
Back	`q.back()`	O(1)	Newest element
Size / Empty	`q.size() / q.empty()`	O(1)	

4. deque — Double-Ended Queue

What it does	Memory	Does NOT support
Like vector but supports O(1) push/pop at both ends. Not contiguous in memory.	O(n)	Slower random access than vector (cache misses). No O(1) middle insert.

```
#include deque dq;
```

Operation	C++ Function	Time	Notes
Push front	`dq.push_front(x)`	O(1)	
Push back	`dq.push_back(x)`	O(1)	
Pop front	`dq.pop_front()`	O(1)	
Pop back	`dq.pop_back()`	O(1)	
Random access	`dq[i]`	O(1)	Slower than vector in practice
Insert middle	`dq.insert(it, x)`	O(n)	
Size / Empty	`dq.size() / dq.empty()`	O(1)	

5. priority_queue — Heap

What it does	Memory	Does NOT support
Binary heap. Max-heap by default: top() is always the LARGEST element. Use greater for min-heap (top = smallest). Does NOT fully sort — only guarantees the top.	O(n)	No arbitrary delete. No search. No update-priority.

```
#include priority_queue pq; // max-heap priority_queue, greater> pq; // min-heap
```

Operation	C++ Function	Time	Notes
Push	`pq.push(x)`	O(log n)	Sifts up the tree
Top	`pq.top()`	O(1)	Max (or min) element
Pop top	`pq.pop()`	O(log n)	Sifts down after removal
Size / Empty	`pq.size() / pq.empty()`	O(1)	

Operation	C++ Function	Time	Notes
Build from array	`priority_queue pq(v.begin(), v.end())`	O(n)	Heapify — faster than n pushes

Memory detail: stores exactly n elements in a plain array — 4n bytes for int. Much lighter than set/map (~40n bytes) because there are no pointers.

6. set — Sorted Unique Elements (Red-Black Tree)

What it does	Memory	Does NOT support
Stores all elements in SORTED order, unique. Every operation traverses a balanced BST of height O(log n). Unlike heap, ALL elements are ordered — not just the top.	O(n) (~40n bytes)	No random access (no operator[]). No duplicates (use multiset).

```
#include set s;
```

Operation	C++ Function	Time	Notes
Insert	`s.insert(x)`	O(log n)	
Delete	`s.erase(x) or s.erase(it)`	O(log n)	By value or iterator
Search	`s.find(x)`	O(log n)	Returns end() if not found
Contains	`s.count(x) – returns 0 or 1`	O(log n)	
Min element	`*s.begin()`	O(1)	Leftmost in tree
Max element	`*s.rbegin()`	O(1)	Rightmost in tree
Lower bound	`s.lower_bound(x)`	O(log n)	First element >= x
Upper bound	`s.upper_bound(x)`	O(log n)	First element > x
Size / Empty	`s.size() / s.empty()`	O(1)	
Iterate in order	`for(auto x : s)`	O(n)	Always sorted

7. multiset — Sorted with Duplicates

What it does	Memory	Does NOT support
Same as set but allows duplicate values. count(x) returns the number of occurrences.	O(n) (~40n bytes)	erase(x) removes ALL copies of x. Use erase(s.find(x)) to remove just one copy.

```
#include multiset ms;
```

Operation	C++ Function	Time	Notes
Insert	`ms.insert(x)`	O(log n)	Duplicates allowed
Erase all copies	`ms.erase(x)`	O(log n + count)	Removes every x
Erase one copy	`ms.erase(ms.find(x))`	O(log n)	Safe way to remove single
Count occurrences	`ms.count(x)`	O(log n + count)	

Operation	C++ Function	Time	Notes
Min / Max	`*ms.begin() / *ms.rbegin()`	O(1)	
Lower/Upper bound	`ms.lower_bound(x)`	O(log n)	Same as set

8. map — Sorted Key-Value Store (Red-Black Tree)

What it does	Memory	Does NOT support
Stores key-value pairs sorted by key. Keys are unique. operator[] inserts a default value if key is absent — be careful.	O(n) (~56n bytes)	No random index access. No duplicate keys (use multimap).

```
#include map mp;
```

Operation	C++ Function	Time	Notes
Insert / Update	`mp[key] = val`	O(log n)	Creates key if absent!
Safe insert	`mp.insert({key, val})`	O(log n)	Won't overwrite existing
Access	`mp[key] or mp.at(key)`	O(log n)	at() throws if key missing
Delete	`mp.erase(key)`	O(log n)	
Search	`mp.find(key)`	O(log n)	Returns end() if not found
Contains	`mp.count(key) – 0 or 1`	O(log n)	
Iterate in order	`for(auto& [k,v] : mp)`	O(n)	Sorted by key
Size / Empty	`mp.size() / mp.empty()`	O(1)	
Lower/Upper bound	`mp.lower_bound(key)`	O(log n)	On keys

9. unordered_set — Hash Set

What it does	Memory	Does NOT support
Hash table. Average O(1) for insert/search/delete. NO ordering — iteration order is arbitrary. Worst case O(n) on hash collisions (rare but possible in contests with anti-hash tests).	O(n) (~several × n bytes for buckets)	No sorted order. No lower_bound/upper_bound. Worst case O(n) with bad hash.

```
#include unordered_set us;
```

Operation	C++ Function	Time	Notes
Insert	`us.insert(x)`	O(1) avg	O(n) worst case
Delete	`us.erase(x)`	O(1) avg	
Search	`us.find(x) / us.count(x)`	O(1) avg	
Size/Empty	`us.size() / us.empty()`	O(1)	

10. unordered_map — Hash Map

What it does	Memory	Does NOT support
Hash table for key-value pairs. Average O(1) for all operations. No ordering.	O(n) (~several × n bytes for buckets)	No sorted order. No lower_bound. Worst case O(n) collisions.

```
#include unordered_map ump;
```

Operation	C++ Function	Time	Notes
Insert / Update	`ump[key] = val`	O(1) avg	Creates key if absent
Access	`ump[key] / ump.at(key)`	O(1) avg	at() throws if missing
Delete	`ump.erase(key)`	O(1) avg	
Search	`ump.find(key)`	O(1) avg	
Contains	`ump.count(key)`	O(1) avg	
Size / Empty	`ump.size() / ump.empty()`	O(1)	

Quick Comparison

Container	Ordered?	Unique?	Insert	Delete	Search	Min/Max	Memory/elem
`vector`	index order	yes	O(1)*	O(n)	O(n)	O(n)	4 B (int)
`stack`	LIFO	yes	O(1)	top only	—	—	4 B
`queue`	FIFO	yes	O(1)	front only	—	—	4 B
`deque`	index order	yes	O(1) ends	O(n) mid	O(n)	O(n)	~4 B
`priority_queue`	heap order	yes	O(log n)	top only	—	O(1) top	4 B
`set`	sorted	yes	O(log n)	O(log n)	O(log n)	O(1)	~40 B
`multiset`	sorted	NO	O(log n)	O(log n)	O(log n)	O(1)	~40 B
`map`	sorted	yes	O(log n)	O(log n)	O(log n)	O(1) key	~56 B
`unordered_set`	NONE	yes	O(1) avg	O(1) avg	O(1) avg	—	~varies
`unordered_map`	NONE	yes	O(1) avg	O(1) avg	O(1) avg	—	~varies

** vector push_back is O(1) amortised — occasionally O(n) when capacity doubles. Memory per element for set/map/unordered containers is approximate and depends on allocator.*